



<https://social.dukla.ch/@andrzej>

Andrzej Wąsowski

Advanced

Programming

Program Correctness & Property-based Testing

IT UNIVERSITY OF COPENHAGEN

SOFTWARE
QUALITY
RESEARCH

Property-Based Testing

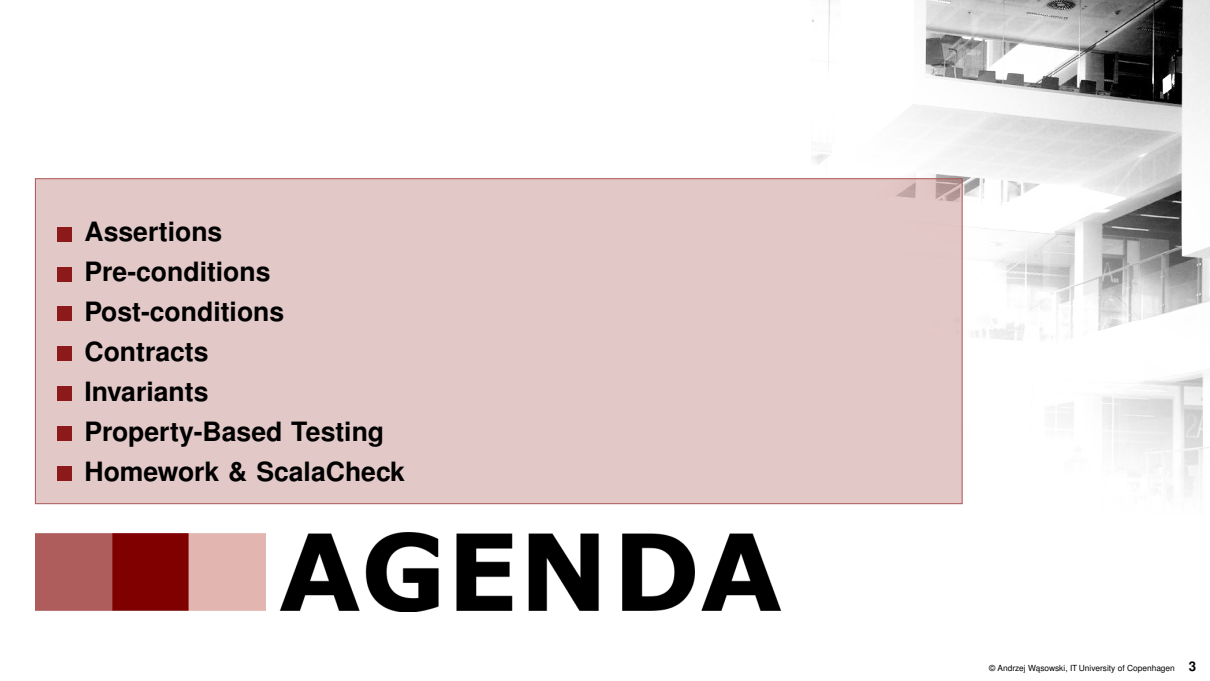
What problem we are solving today?

The Problem

- Scenario testing (unit testing) tries only the situations for which you created test-cases, and which you anticipated
- This **increases the cost** of testing (you need to create many test-cases)
- This **lowers the coverage** of tests (only things you anticipated are tried)

The Solution

- **Automate** generation of test data
- Use **randomized inputs**, generated in controlled manner (which we know how to do from the State chapter)
- **Generalize** tests to properties that should hold on many possible inputs

- 
- **Assertions**
 - **Pre-conditions**
 - **Post-conditions**
 - **Contracts**
 - **Invariants**
 - **Property-Based Testing**
 - **Homework & ScalaCheck**



AGENDA

Assertions

```
1 // assertion: an empty list has size zero
2 assert(List().size == 0)

4 def f(l: List[Int]): Unit =
5   // A contract for Cons: If I have a list, and add an element to it
6   // it will certainly have at least one element in it
7   assert(Cons(42, l).size > 0)

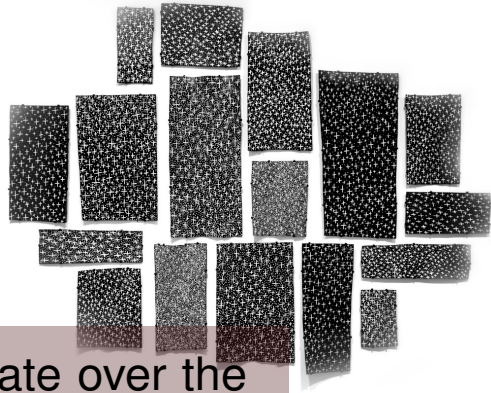
9   // A contract for Cons: If length of a list is n, and you add (Cons)
10  // an element to the list, it will have size n+1
11  assert(l.size + 1 == Cons(42, l).size)

13  // A contract for Cons: If you add (Cons) an element to the list,
14  // then this element will be the head of the list.
15  assert(Cons(42, l).headOption == Some(42))

17 def head(l: List[Int]): Int =
18   // Precondition for head: l is not empty
19   assert( ... ) // <----- menti: 6887 0869
```

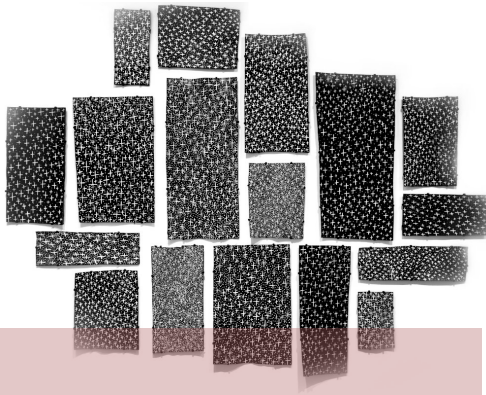
- Analyze **examples**
- **English** turned into **predicates** over program state (variables, values)
- Invariants, pre-conditions, post-conditions, contracts
- In Scala: `assert` is a **method** on the `Predef` object, automatically imported
- 2-argument version: produce an error msg

Definition



Assertion is a Boolean predicate over the program state (variables and values), usually expressed as a logical proposition, that evaluates to true in the correct executions.

Why Assertions?



- Help the programmer read the code
- Facilitate fail-fast programming
- Help compilers compile efficiently
- Support **testing** and **verification**

Flavours of Assertions

Scala terminology

- **Require**: before an action, bug if violated, **blame the caller**, a **pre-condition**
- **Ensuring**: after an action, bug if violated, **blame the callee**, a **post-condition**;
In Scala it is an expression: `(e) ensuring { x => P(x) }`, returns the value of `e`
- **Assert**: Must hold, the checker should prove it, a goal, bug if does not hold; A test
- **Assume**: Consider only satisfying executions; Verifier assumes this; An axiom
- At runtime they all just **fail** with different exceptions, if violated
- Differences mostly visible in interpreting the failure (and in formal verification)
- Assertions are **not** specific to imperative programming, used in FP too
- Write **small assertions** if possible, test a single violation at a time. Gives better feedback.

Pre-conditions for Option

```
1 def isEmpty: Boolean = this match { case None => true; case _ => false }
2 // Get the value held by an option
3 // Precondition for 'get': !this.isEmpty
4 def get: A = ...

6 // Apply a binary operator to all elements of this iterable collection
7 // Like foldRight, but with an implicit initial element. Examples:
8 // List(1,2,3).reduce[Int] { _+_ } == 6
9 // List(1).reduce[Int] { _+_ } == 1
10 // Some(1).reduce[Int] { _+_ } == 1
11 // Nil.reduce[Int] { _+_ } == 0
12 // precondition for 'reduceRight': true
13 def reduceRight[B >: A](op: (A, B) => B): B

15 // Count the number of elements in a collection that satisfy a predicate
16 // Task: what is the precondition for 'count' in terms of this 'object'?
17 def count(p: A => Boolean): Int           // 6887 0869

19 // Always fail with an exception
20 // Task: what is the precondition for this.f to avoid failure?
21 def f = throw Exception()               // 6887 0869
```

- **Pre-condition true**
means
always succeeds
(no pre-condition)
- **Pre-condition false**
means
never succeeds
(always fail)

Pre-conditions (require)

Pre-condition: A Boolean **predicate** such that **if it holds before execution of some code** then the code behaves correctly.

- Typically considered at the **entry point to a function** or procedure
- Can constrain **arguments** + other **variables** and **values** in scope
- **Enforced using assertions** in most programming languages
- A function can have **several pre-conditions** (a single example)
 - Consider: `def factorial(n: Int): Int`
 - $n \geq 0$ is a pre-condition
 - $n \geq 2$ is a pre-condition, too. **Why?**
 - $n \geq 0$ is weaker, $n \geq 2$ is stronger. **Why?**
- The **weakest pre-condition**: the minimal assumption for a block of code (typically a function) to behave correctly, so to satisfy the expected post-condition
- We specify the weakest pre-conditions for functions to achieve **complete specifications**
- **Stronger pre-conditions** are used in testing (when testing only one aspect of logics)

Post-conditions

```
1 def size[A](t: Tree[A]): Int
2 // A post-condition: forall t. size(t) >= 1

4 def depth[A](t: Tree[A]): Int
5 // A post-condition: forall t. depth(t) >= 0

7 // A helper function
8 def get(t: Tree[Int]): Int = t match
9   case Leaf(n) => n
10  case Branch(l, _) => get(l)

12 // Now we use it
13 def maximum(t: Tree[Int]): Int
14 // A post-condition: forall t: maximum(t) >= get(t)

16 // We previously defined
17 def nextInt: (Int, RNG) = ...
18 // The postcondition: true
19 // Not the strongest post-condition for random,
20 // but it is hard to give a stronger one.
21 // This would require formally defining the pseudo-
22 // random stream (likely not needed)
```

- Are these the **strongest post-conditions** we could think of? Could we guarantee more?
- Strongest post-conditions may be **difficult** to write
- The maximum post-condition **relates** more than one function (we shall return to this)
- A **true** post-condition means that we cannot say anything useful about the result
- (like a false pre-condition meant that there is no useful conditions in which the function could be called)

Post-conditions (ensuring)

Post-condition A Boolean **predicate** describing a correct result of an execution of a fragment of code.

- Typically **applied to functions**
- For a function it **constraints the return value**
- In imperative programs also **constraints variables and values in scope**
- The **strongest post-condition**: the maximum guarantee for a code fragment given by a correct execution, so an execution that satisfies a pre-condition
- Strongest post-conditions are used to define the **complete behavior** of code
- **Testing**: check post-conditions with assertions after a function call
- Giving strongest post-conditions is often difficult without replicating the code
- **Decompose the post-condition** and test for several weaker conditions instead

Contracts

```
1 // Returns this scala.Option if it is nonempty, otherwise return
2 // the result of evaluating alternative.
3 def orElse[B >: A](alternative: => Option[B]): Option[B]
4 // pre-condition: true
5 // post-condition:
6 //   (!this.isEmpty => orElse(alternative) == this      ) &&
7 //   ( this.isEmpty => orElse(alternative) == alternative)

9 def maximum (t: Tree[Int]): Int
10 // pre-condition: true
11 // post-condition:
12 //   (forall a in t: { a <= maximum(t) }) &&
13 //   (exist  a in t: { a == maximum(t) })
14 // (forall/exist could be implemented for trees as static methods)

16 // For partial functions a pre-condition is not true.
17 def get[A](oa: Option[A]): A = ...
18 // pre-condition: !oa.isEmpty
19 // post-condition: Some(get(oa)) == oa
```

- We are using **algebraic laws** again in all three examples
- A key aspect of correct behavior is that it **interacts well** with other parts of our API

Contracts

Contract. A contract for a function is a **pair of a pre-condition and a post-condition**. The caller of the function must ensure that the pre-condition is satisfied at the call time, and the callee has to ensure that the post-condition holds after termination, **if** the pre-condition was satisfied.

- Ideally a contract **minimizes assumptions** and **maximizes guarantees** (the pre-condition is the weakest, the post-condition is the strongest)
- Writing these ideal complete contracts may be difficult
- In testing, we can use **weaker contracts**:
 - If ϕ holds before the call then ψ holds after the call
 - Not necessarily the weakest/strongest
 - We decompose the ideal contract into tests and test-cases, or, in other words pairs of stronger preconditions and weaker post-conditions
- **NB.** The weakest contract is $\text{false} \Rightarrow \text{true}$
 - Any terminating function satisfies it vacuously

Invariants

- An **invariant** is a related concept: A property that should always hold (for an object/concept at runtime)
- **Loop invariant** holds at every loop iteration (not useful in FP). Why?
 - Insertion sort: Elements left of the current index form a sorted sequence
- In FP loop invariants are replaced by contracts of recursive functions
- **Data structure invariant** holds always for instances of the data structure:
 - AVL trees: The difference of height between the left and right subtree is at most 1 for any AVL tree node

Property-based testing in a nutshell /1

Actual test code from exercises in the prior weeks

Three weak/partial contracts/laws for map on trees
(with trivial/true preconditions)

```
1 property("Ex04.02: identity is a unit with map") =  
2   forAll { (t: Tree[Int]) => Tree.map(t)(identity[Int]) == t }  
  
4 property("Ex04.03: map does not change size") = // 6887 0869  
5   forAll { (t: Tree[Int], f: Int => Int) =>  
6     Tree.size(Tree.map(t)(f)) == Tree.size(t) }  
  
8 property("Ex04.04: map is 'associative'") = // 6887 0869  
9   forAll { (t: Tree[Int], f: Int => Int, g: Int => Int) =>  
10    Tree.map (Tree.map(t)(f))(g) == Tree.map(t)(g compose f) }
```

- forAll generates random Tree[Int]
- forAll checks if the predicate (lambda) holds for all random data. Fail if false
- This code uses the **ScalaCheck** library
- We implement a similar lib next week

This code can be found in 030-option/Exercises.test.scala

Property-based testing in a nutshell /2

Test-data generators

```
1 def genTree[A](using arbA: Arbitrary[A]): Gen[Tree[A]] =  
2   for  
3     coin <- Gen.frequency(7 -> true, 1 -> false)  
4     tree <- if coin  
5       then arbA.arbitrary.map(Leaf.apply)  
6       else genTree.flatMap { l => genTree.map { Branch(l, _) } }  
7   yield tree  
  
9 given arbitraryTree: [A: Arbitrary] => Arbitrary[Tree[A]] =  
10  Arbitrary[Tree[A]] (genTree)
```

- This week we learn how to **use** the API
- Next we **build** such API, incl. givens + Gen monad
- When you test a new data type: **create a generator** for it
- Normally you write much **more tests** than generators

- genTree generates a random tree of A, if it **can find** a given generator of arbitrary A
- Gen.frequency is a Gen[Boolean], returns true/false with relative frequencies 7/1 (polymorphic)
- arbA.arbitrary is a Gen[A], **mapped** into a leaf containing the A (Gen[Leaf[A]])
- Last 2 lines expose a **given** generator of arbitrary trees of As, so forAll and other Gens can find it

This code can be found in `030-option/Exercises.test.scala`

Scenario tests vs Property tests

- Most traditional unit-level and integration-level testing uses **scenarios**
- Examples, test cases, **one path through a system** per test
- Scenarios are **obtained from requirements**
- Good for **covering requirements**.
- Good for testing **special corner cases**
- Good for recording **regression tests**
- **Automation key** for success
- Test libraries are more important than debuggers, in the sense that if you automate tests, you **reuse the effort**
- BDD, TDD, JUnit (XUnit),
Cucumber/RSpec, scalatest

- **Property-based testing (PBT)** tests **algebraic laws** that should hold for an API
- The process is:
 - 1 Formulate laws
(pre/post/invariant thinking helps)
 - 2 Generate random data
 - 3 Test the laws on this data
- Gives **better** (? rather alternative) **coverage** than scenario testing
- It may catch things that you **did not predict**, due to randomness
- Gets closer to **verification** but remains easy
- You use **formal specifications** to test

PBT and Contracts

```
1 // Recall the specification      // pre-condition: !oa.isEmpty
2 def get[A](oa: Option[A]): A     // post-condition: Some(get(oa)) == oa

4 // When testing in the classical way, create a value 'oa', run get, inspect the result
5 property ("test get contract") = // Example with ScalaCheck simple Boolean property
6   val oa = Some(1)              // (like an assertion)
7   get (oa) == 1                  // Any Boolean function (predicate) is treated like an assert

9 // Two weaknesses:
10 // 1. For another integer, write another test; create repetitive code (or iterate a collection)
11 // 2. The contract specification is not explicit in the test.
12 // A property-based test does the iteration implicitly and solves these problems:

14 property ("test get contract (PBT)") =
15   forAll { (oa: Option[Int]) =>           // <-- quantifier
16     !oa.isEmpty ==>                       // <-- pre-condition (implication)
17     Some(get(oa)) == oa }                 // <-- post-condition

19 // An explicit contract; 100 different tests without test case tables and iterations
```

Homework / Exercises

- The goal is to **test our lazy list library**
- We invert the situation from previous weeks: assume that the type (LazyList) is implemented, you write the test file
- The build system is **set up with the testing library**, and 3 example tests
- There is even an **example generator** of lazy lists
- **Add more tests**, and possibly more generators
- There must be **several property-based tests** (PBT) in your solution to pass. The rules are explained in the Exercise file (slightly different than other weeks)

Resources on ScalaCheck

- ScalaCheck: <https://github.com/typelevel/scalacheck/blob/main/doc/UserGuide.md>
- Gen, a key type: https://javadoc.io/doc/org.scalacheck/scalacheck_3/1.17.0/org/scalacheck/Gen.html
- Arbitrary, a key type: https://javadoc.io/doc/org.scalacheck/scalacheck_3/1.17.0/org/scalacheck/Arbitrary.html.
- I also find source code at <https://github.com/typelevel/scalacheck/blob/v1.17.0/core/shared/src/main/scala/org/scalacheck/Gen.scala> useful to read (actually I read it more often than the docs).
- Additionally, next week we study the design of Prop and Gen, to deepen understanding (you can start reading Ch. 8)

Further reading: Blogs! (Read one for context)

- **C++:** couldn't find a nice blog post; read the user guide of rapidcheck if C++ is your fare: https://github.com/emil-e/rapidcheck/blob/master/doc/user_guide.md
- **C#:** <https://www.codit.eu/blog/property-based-testing-with-c/> with <https://fscheck.github.io/FsCheck/>
- **F#:** <https://www.codit.eu/blog/practically-property-based-testing-your-strict-domain-model-in-f-c/> using <https://fscheck.github.io/FsCheck/>
- **Go:** <https://earthly.dev/blog/property-based-testing/> uses a small CSV file manipulation tool as an example
- **Haskell:** <https://www.fpcomplete.com/blog/2017/01/quickcheck/> with Quickcheck, of course!
- **Java:** <https://yoan-thirion.medium.com/improve-your-software-quality-with-property-based-testing-70bd5ad9a09a> with junit-quickcheck. There is also <https://jqwik.net> which seems to be a rich library used also in Kotlin and Groovy
- **JavaScript/TypeScript:**
<https://mokkapps.de/blog/property-based-testing-with-type-script> using <https://github.com/dubzzz/fast-check>
- **Kotlin:** <https://instil.co/blog/from-tdd-to-pbt-via-kotest/>
Uses <https://kotest.io> which apparently also runs on non-jvm tool chains of Kotlin.
- **Python:** <https://medium.com/clarityai-engineering/property-based-testing-a-practical-approach-in-python-with-hypothesis-and-pandas-6082d737c3ee> and <https://datascience.blog.wzb.eu/2019/11/08/property-based-testing-for-scientific-code-in-python/>, the latter with more data-science angle, both using Hypothesis.
- **Rust:** with <https://blog.auxon.io/2021/02/01/effective-property-based-testing/> with proptest, and <https://dev.to/itminds/introduction-to-property-based-testing-via-rust-3h6f> with quickcheck
- **Scala:** <https://medium.com/analytics-vidhya/property-based-testing-scalatest-scalacheck-52261a2b5c2c>, shows how to PBT within ScalaTest (the unit testing framework for Scala)